CSE 144 Final Project Report
Transfer Learning Challenge
Group Members (max 3):
Ben Liu (CruzID: 1932243)
Milan Moslehi (CruzID: 1977483)
Aaron Yam (CruzID: 2105044)
Spring 2026

---

1 Introduction

1. Problem goal and setting.
We address a 100-way fine-grained image classification task with roughly 10 labeled training images per class and 1,036 unlabeled test images. The goal is to train a model that generalizes from this small labeled set and produce a submission.csv of predicted class labels (0–99) for Kaggle evaluation.
2. Why transfer learning is appropriate.
With only ~1,000 training images across 100 classes, training a convolutional network from scratch would severely overfit and fail to learn useful visual features. ImageNet-pretrained models already encode low- and mid-level representations (edges, textures, object parts) that transfer well to new visual domains. Fine-tuning a pretrained backbone with a task-specific classifier head is the standard and most effective approach for low-data image classification.
3. Brief summary of your approach and main result.
We use EfficientNet V2-S pretrained on ImageNet, replace the default classifier with a deeper head (512-d hidden layer, SiLU activation, dropout), and train in two phases: (1) frozen backbone, classifier-only warmup for 5 epochs; (2) full fine-tuning for up to 45 epochs with early stopping. We apply moderate data augmentation, label smoothing, discriminative learning rates, and 4-view test-time augmentation (TTA) at inference. On our stratified validation set (1 image per class), the best checkpoint achieves 84.0% validation accuracy at epoch 23 (global epoch 28). Final predictions are generated with TTA and saved to submission.csv.

---

2 Dataset

1. Number of classes and dataset sizes (train/val/test).
- Classes: 100 (labels 0–99)
- Training: 1,000 images total (10 per class)
- Validation: 100 images (1 per class, held out via stratified split)
- Effective training set after split: 979 images
- Test: 1,036 images (no labels provided)

2. Directory structure and label mapping details (ensure labels 0–99 match folders).
Data lives under ucsc-cse-144-spring-2026-final-project/:
train/
 0/   *.jpg

1/ *.jpg
...
99/ *.jpg
test/
0.jpg, 1.jpg, ..., 1035.jpg
Class labels are taken directly from folder names: folder 42/ maps to label 42. Only numeric subdirectories are used. Images are loaded as RGB JPEGs via PIL.
3. Preprocessing (resize, normalization) and data augmentation used.
Training transforms:

- Resize to 340×340 (IMG_SIZE + 40)
- RandomResizedCrop to 300×300, scale (0.82, 1.0), aspect ratio (0.88, 1.14)
- RandomHorizontalFlip (p = 0.5)
- ColorJitter (brightness 0.12, contrast 0.12, saturation 0.08)
- ToTensor + ImageNet normalization (mean [0.485, 0.456, 0.406], std [0.229, 0.224, 0.225])

Evaluation transforms:

- Resize to 340×340, CenterCrop to 300×300
- Same normalization as training

Test-time augmentation (inference): 4 views — standard center crop, horizontal flip, and two slightly larger resize/crop scales (332 and 348) with and without flip. Softmax probabilities are averaged across views before argmax.

---

3 Implementation

3.1 Model

1. Pretrained backbone used (e.g., ResNet/EfficientNet/ViT) and why.
We use EfficientNet V2-S with EfficientNet_V2_S_Weights.IMAGENET1K_V1 weights from TorchVision (~20.9M parameters). EfficientNet V2 improves training efficiency and accuracy over V1 while remaining compact enough to fine-tune on Apple Silicon (MPS). It balances capacity and regularization better than smaller models (e.g., ResNet-18) for a 100-class problem with limited data, without the memory cost of larger ViTs.
2. Architecture changes (classifier head, pooling, dropout, etc.).
The default single linear classifier is replaced with:
Dropout(p=0.3)
→ Linear(in_features → 512)
→ SiLU
→ Dropout(p=0.2)
→ Linear(512 → 100)
The convolutional feature extractor is unchanged. The custom head adds capacity for 100-way discrimination while dropout limits overfitting on the small dataset.
3. Fine-tuning strategy (frozen layers, unfreezing schedule).

Phase 1 (5 epochs): Freeze entire backbone; train only the classifier head (~707K trainable parameters). This lets the new head adapt to class boundaries before backbone weights shift. Phase 2 (up to 45 epochs): Unfreeze all parameters (~20.9M trainable). Use a lower learning rate on the backbone and a 3× higher rate on the classifier (discriminative fine-tuning). Early stopping with patience = 10 epochs on validation accuracy prevents overfitting.

3.2 Training

1. Loss function and optimizer.
   - Loss: CrossEntropyLoss with label smoothing = 0.03
   - Optimizer: AdamW with weight decay = 7e-4
   - Gradient clipping: max norm = 1.0
   - Mixup: implemented but disabled (MIXUP_ALPHA = 0.0) based on validation performance
2. Learning rate, scheduler, batch size, epochs, weight decay.

Setting	Value
Image size	300
Batch size	12
Phase 1 epochs	5
Phase 1 LR	8e-4 (OneCycleLR, pct_start=0.2)
Phase 2 epochs	up to 45 (early stop at 33)
Phase 2 backbone LR	6e-5
Phase 2 classifier LR	1.8e-4 (3× backbone)
Phase 2 scheduler	CosineAnnealingLR, η_min = 8e-7
Weight decay	7e-4
Early stopping patience	10

3. Hardware/software environment (GPU, PyTorch version).
   - Device: Apple MPS (Metal Performance Shaders)
   - PyTorch: 2.8.0
   - TorchVision: EfficientNet V2-S pretrained weights
   - Other: NumPy, Pandas, PIL, tqdm
   - DataLoader workers: 0 (for reproducibility on macOS)

---

4 Experiments

1. Baseline setup.

Initial baseline: EfficientNet V2-S with default classifier, frozen backbone for 5 epochs, then full fine-tuning with a single global learning rate. This established that transfer learning alone reaches ~38% validation accuracy after Phase 1 and ~49% after the first Phase 2 epoch.
2. Hyperparameter tuning plan and validation method.
We used a stratified validation split — exactly 1 image per class (100 images), seeded with SEED=43 — so every class is represented in validation. Hyperparameters were tuned manually by monitoring validation accuracy and loss:

- Increased image size from 224 → 300 for finer detail
- Reduced batch size to 12 for stability on MPS
- Added custom classifier head with dropout
- Tuned Phase 1/2 learning rates and introduced discriminative LR (3× on classifier)
- Added label smoothing (0.03) and weight decay (7e-4)
- Set early stopping patience to 10
- Enabled 4-view TTA for final submission

3. Ablations (augmentations, model size, freezing strategy, LR, etc.).

Change	Observation
Two-phase training (freeze → unfreeze)	Stable warmup; val acc rose from 13% → 38% in Phase 1 alone
Custom 512-d classifier head + dropout	Better val generalization vs. default linear head
Discriminative LR (classifier 3× backbone)	Faster head adaptation without destabilizing pretrained features
Label smoothing (0.03)	Reduced overconfident predictions on small val set
Mixup (disabled, α=0)	Left in code but turned off; did not help validation
Larger input (300px)	Improved fine-grained discrimination
TTA (4 views)	Used at inference to reduce variance on test predictions
Early stopping (patience=10)	Stopped at epoch 33; best checkpoint at epoch 23 (84% val)

---

5 Results

1. Training/validation accuracy (and loss) and any key curves/plots (optional).
Phase 1 (classifier only, 5 epochs):

Epoch	Train Acc	Val Acc
1	7.2%	13.0%
5	50.7%	37.0%

Phase 2 (full fine-tune, selected epochs):

Epoch	Train Acc	Val Acc
1	51.7%	49.0%
5	90.2%	73.0%
12	99.7%	80.0%
19	99.8%	82.0%
21	99.8%	83.0%
23	99.9%	84.0% (best)
33	99.9%	83.0% (early stop)

Training accuracy reached ~100% by epoch 24 while validation accuracy plateaued at 84%, indicating mild overfitting — consistent with only 979 training images. The gap between train (~100%) and val (84%) suggests the model memorizes training samples but still generalizes reasonably on held-out images.
Best checkpoint: epoch 28 (5 Phase 1 + 23 Phase 2), validation accuracy 0.8400, validation loss 0.844.
2. Kaggle public leaderboard score (and submission settings).

- Submission file: submission.csv (1,036 rows, columns ID, Label)
- Inference settings: best checkpoint loaded from checkpoints/best_model.pt, 4-view TTA enabled, labels in range [0, 99]
- Kaggle public leaderboard score:85%

3. Brief qualitative examples or error analysis highlights (optional).
With only 1 validation image per class, a single misclassification per class heavily affects the 84% figure — each error costs 1%. Classes with visually similar instances (shared color palettes, backgrounds, or fine-grained species differences) are the most likely failure cases. Training accuracy near 100% with validation stuck at 84% suggests the model benefits from more diverse augmentation or additional regularization for the hardest classes.

---

6 Discussion

1. What worked best and why.
The combination of EfficientNet V2-S pretraining, a deeper dropout-regularized classifier head, and two-phase discriminative fine-tuning worked best. Pretraining provides strong generic features; Phase 1 stabilizes the new head; Phase 2 adapts low-level features at a conservative backbone LR. Label smoothing and weight decay further reduced overfitting on the tiny dataset. TTA at inference provides a free boost by averaging predictions over spatial and flip variants.
2. Failure cases, overfitting/underfitting observations.
Clear overfitting emerges after ~epoch 12: training accuracy exceeds 99% while validation accuracy oscillates between 78–84%. Early stopping at epoch 33 correctly halted training before further degradation. Failure modes likely include: (a) classes with high intra-class visual

variance across the 10 training images, (b) classes confused with visually similar neighbors, and (c) validation images that differ in lighting or framing from the training distribution.
3. Limitations and concrete next improvements.

- Very small training set: 10 images/class limits ceiling performance; more data or pseudo-labeling could help.
- Tiny validation set: 1 image/class makes val accuracy noisy; k-fold cross-validation would give more reliable estimates.
- Single model: An ensemble of EfficientNet V2-S/M or a ViT-Small could improve robustness.
- Stronger augmentation: RandAugment, CutMix, or Mixup with tuned α may improve generalization.
- Longer backbone warmup: Unfreezing only the last feature block before full fine-tune (partial unfreezing) could reduce overfitting.
- Larger input / stronger TTA: 384px inputs with more TTA scales may help fine-grained classes.

---

7 Reproducibility

1. Random seeds and determinism settings.
SEED = 43
os.environ["PYTHONHASHSEED"] = str(SEED)
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.backends.cudnn.benchmark = False
torch.backends.cudnn.deterministic = True
Validation split uses random.Random(SEED) for per-class shuffling. DataLoader uses num_workers=0 for deterministic behavior on macOS.
2. Package versions / environment setup steps.

1. Clone the repository and place the dataset under ucsc-cse-144-spring-2026-final-project/.
2. Install dependencies:

pip install torch torchvision numpy pandas pillow tqdm certifi

3. Verified environment: PyTorch 2.8.0, Python 3.x, MPS or CUDA device.

3. Exact commands to train and to generate submission.csv.
Open and run all cells in train.ipynb, or equivalently:
jupyter notebook train.ipynb
Run all cells sequentially
The notebook will:

1. Train Phase 1 (5 epochs, frozen backbone)
2. Train Phase 2 (up to 45 epochs with early stopping)
3. Save best model to checkpoints/best_model.pt
4. Load best checkpoint, run TTA inference, and write submission.csv

https://drive.google.com/file/d/1z9NXb1m_oRLn3R8NAWz_Q6SRgEus2f-p/view?usp=sharing
Link to weights in Google Drive, if it does not work you can view in the github repo from checkpoints->best_model.pt

For further reference ins cell block 2 in train.ipynb:

```
SEED = 43

IMG_SIZE = 300
BATCH_SIZE = 12
NUM_CLASSES = 100
NUM_WORKERS = 0

PHASE1_EPOCHS = 5
PHASE2_EPOCHS = 60

PHASE1_LR = 8e-4
PHASE2_LR = 6e-5

WEIGHT_DECAY = 7e-4
LABEL_SMOOTH = 0.03
MIXUP_ALPHA = 0.0

PATIENCE = 14
```

---

8 Team Contributions

1. Ben: Training pipeline implementation, hyperparameter tuning
2. Milan: Model architecture design and classifier head modifications
3. Aaron: Data augmentation strategy and dataset preprocessing

---

9 References

1. TorchVision documentation / pretrained model sources.
    - TorchVision EfficientNet V2 models: https://pytorch.org/vision/stable/models/efficientnetv2.html
    - EfficientNet_V2_S_Weights.IMAGENET1K_V1 pretrained weights
2. Any papers or tutorials used.
    none